DSA 2020A: ARTIFICIAL INTELLIGENCE

Lab 2 Assignment: Building a Multi-Agent AI System for Real-World Applications

Group Size: Individual or pairs (strongly encouraged for pairs to enable better brainstorming of roles, coordination strategies, and debugging of inter-agent interactions)

Estimated Time: 2 lab sessions.

Learning Objectives

By completing this assignment, students will be able to:

- Understand the motivation, benefits, and challenges of **multi-agent systems** compared to single-agent approaches
- Design specialized roles for multiple AI agents that collaborate toward a shared goal
- Implement a multi-agent architecture using an open-source framework (with focus on coordination, communication, and orchestration)
- Handle inter-agent handoffs, shared memory/state, conflict resolution, and termination conditions
- Integrate tools/APIs and incorporate **human-in-the-loop** oversight
- Reflect on ethical considerations, reliability, and real-world applicability
- Evaluate agent team performance and identify coordination bottlenecks

Core Concept

Modern AI applications increasingly rely on multi-agent systems (MAS) , teams of specialized agents that divide labor, critique each other, iterate collaboratively, and achieve more complex, reliable outcomes than a single generalist agent. This lab shifts from building one autonomous agent to designing an **agent team** that mirrors real organizationals (division of labor).

Use Case Options (Choose ONE – must implement as a multi-agent team)

Select one domain and transform the original single-agent idea into a **collaborative multi-agent workflow.** All options require at least **3–5 specialized agents** working together under orchestration.

1. **Customer Care Team** Goal: Handle complex customer inquiries end-to-end (beyond simple Q&A). Typical agents:
    - Greeter / Intent Classifier
    - Researcher / Knowledge Retriever (searches company docs/FAQ/web)
    - Empath / Tone Adapter
    - Resolver / Action Taker (e.g. refund via API, schedule callback)
    - Escalator / Human Handoff Coordinator
    - Quality Reviewer (post-resolution critique)

2. **Healthcare Advisory Team** (with strong disclaimers: not medical advice) Goal: Interactive symptom/discussion assistant that gathers info safely and responsibly. Typical agents:
    - Intake / History Collector
    - Symptom Analyst / Differential Diagnoser
    - Evidence Gatherer (searches reliable sources)
    - Risk & Safety Checker (flags red flags, ensures disclaimers)
    - Recommendation Formatter & Educator
    - Follow-up Planner
3. **Personalized Education / Tutoring Team** Goal: Adaptive tutor for a chosen subject/topic (e.g. Python, high-school algebra, history). Typical agents:
    - Curriculum Planner
    - Concept Explainer
    - Quiz / Assessment Generator
    - Feedback Analyzer & Misconception Detector
    - Motivator / Engagement Booster
    - Progress Tracker & Report Generator
4. **Personal Finance Assistant Team** Goal: Help users create/manage budgets, analyze spending, suggest optimizations. Typical agents:
    - Data Ingestion / Categorizer (parses user input/transactions)
    - Budget Forecaster / Calculator
    - Expense Pattern Detector
    - Savings & Investment Advisor
    - Risk & Compliance Checker
    - Summary & Visualization Generator
5. **Entertainment Recommendation & Curation Team** Goal: Build rich, personalized entertainment plans (movies, series, music, books, games). Typical agents:
    - Preference Profiler / Conversation Guide
    - Content Researcher / Metadata Gatherer
    - Taste Matcher / Similarity Engine
    - Diversity & Serendipity Curator
    - Schedule / Plan Builder
    - Feedback Learner & Refiner

Required Technical Elements (All Options)

- Use one of the following open-source multi-agent frameworks :
    - **LangGraph** (strongly recommended – hierarchical/supervisor pattern, excellent state control)
    - **CrewAI** (role/task-centric, easiest for beginners)
    - **Microsoft AutoGen** (conversational style)
    - **OpenAI Swarm** (lightweight – good for experimentation)
- At minimum implement:
    - **Supervisor/Orchestrator** agent (decides who acts next, when to stop, when to ask human)

- 3–5 **specialized worker agents** with distinct roles & prompts
- **Shared state/memory** (query history, findings, user profile, intermediate results)
- **Tool integration** (at least 2–3 tools total, e.g. web search, calculator/code exec, file read, calendar API simulation)
- **Human-in-the-loop** capability (pause for approval on sensitive actions/plans)
- **Reflection/critique loop** (at least one agent critiques outputs before final delivery)
- **Streaming** of agent thoughts/actions for visibility
- **Termination condition** (goal achieved, max turns, user stop, etc.)

Deliverables

1. GitHub repository containing:
    - Main implementation file(s)
    - requirements.txt
    - README.md with:
        - Chosen use case & rationale
        - Agent team diagram (text or image: roles + communication flow)
        - How to run (including LLM provider setup)
        - 2–3 example interaction transcripts showing collaboration
        - Key challenges (coordination, hallucinations, loops, cost) & solutions
2. Short demo (Jupyter notebook + screenshots) showing the team handling a realistic user scenario
3. Reflection report (2-3 paragraphs):
    - What advantages did the multi-agent approach provide vs. a hypothetical single agent?

Grading Rubric (100 points)

- Clear role specialization & meaningful division of labor (20)
- Effective orchestration / supervisor logic (20)
- Tool use & integration across agents (15)
- Shared state/memory & inter-agent coordination (15)
- Human-in-the-loop & safety/reflection mechanisms (10)
- Code quality, structure, documentation (10)
- Demo quality (5)
- Depth of reflection & insight (5)

Recommended Resources

- LangGraph multi-agent tutorials (hierarchical supervisor pattern)
- CrewAI quickstart & role/task examples
- AutoGen conversational multi-agent notebooks
- DeepLearning.AI short course: "Multi AI Agent Systems with crewAI"
- Ethical AI guidelines for your chosen domain